# Enterprise Customer Onboarding Agent — Solution Design Document

April 2026

## 1. Architecture Overview

The solution implements an **AI-powered automation agent** that orchestrates enterprise customer onboarding from deal closure through SaaS provisioning. Built with **Pydantic AI** for agentic tool calling and **FastAPI** for the REST interface, the agent uses native LLM tool calling to autonomously reason through 28 registered tools. Each tool is mirrored as a **FastMCP** server definition for MCP-style extraction. A second **CS Assistant Agent** shares all tools but returns free-form text for conversational interaction via a **Streamlit UI** with four pages (Dashboard, Portfolio Overview, Run Onboarding, Chat). Data flows sequentially across systems via **cross-system chaining**: Opportunity.ContractId → SF Contract.Id → CLM.salesforce_contract_id → Invoice.clmContractRef, with fallback to account-based lookups when chain links are missing. The latest prototype also closes the CS action loop: suggested actions can simulate upstream issue resolution in mocked SaaS systems and then re-run onboarding so a previously blocked or escalated account can actually progress into provisioning.

### 1.1 System Integrations

| System | Objects & Key Fields |
|---|---|
| **Salesforce CRM** | `Account` (Id, Name, IsDeleted, OwnerId), `Opportunity` (StageName, Amount, **ContractId**), `Contract` (**Id**, Status, StartDate), `User` (IsActive, Email). [API Ref] |
| **CLM System** | contract_id, status, **salesforce_contract_id** (chains from SF Contract.Id), signatories, key_terms. Mock simulating DocuSign CLM. |
| **NetSuite ERP** | `Invoice` (status, dueDate, amountRemaining, total, **clmContractRef** chains from CLM contract_id). [API Ref] |
| **Provisioning** | Tenant creation + 14-task onboarding checklist with dependencies, owners, and due dates. |
| **Currency (Frankfurter)** | Live and historical exchange rates via ECB data. Converts invoice amounts across currencies (e.g., CAD↔USD) using the invoice date rate. [API Ref] |
| **Sentiment (ML)** | DistilBERT transformer model (`distilbert-base-uncased-finetuned-sst-2-english`) via PyTorch + HuggingFace for customer interaction scoring. Keyword fallback when ML stack unavailable. |

**Field Selection Rationale:** `Account.IsDeleted` validates account exists; `Opportunity.StageName="Closed Won"` confirms deal closure; `Invoice.status` identifies payment blockers; `CLM.status` in {EXECUTED, SIGNED} required for provisioning. Currency conversion uses historical rates from the invoice date for financial accuracy. Minimizes API calls while capturing decision-critical data.

### 1.2 Data Flow

**(1)** Webhook with `account_id` + `correlation_id` **(2)** Fetch account, then user/opportunity in parallel; chain sequentially: Opp.ContractId → SF Contract → CLM (via salesforce_contract_id) → Invoice (via clmContractRef) **(3)** `validate_business_rules` + `check_financial_alignment` **(4)** PROCEED → provision + notify; BLOCK/ESCALATE → notify, remediate, and optionally re-run **(5)** PROCEED only: mandatory `check_onboarding_progress` + `identify_onboarding_risks` + `get_customer_sentiment` for post-provisioning risk and sentiment assessment.

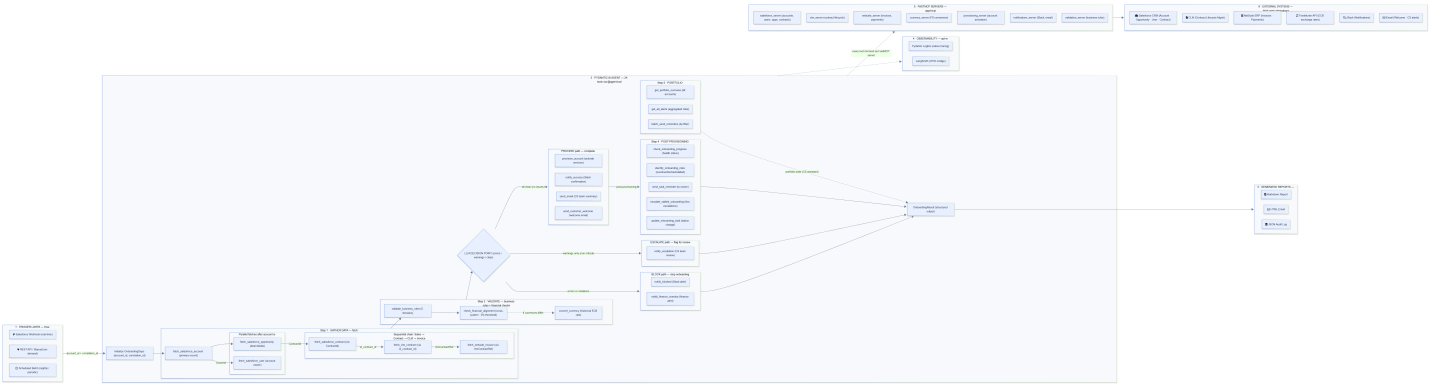

**Figure 1:** Architecture: triggers, Pydantic AI agent orchestration with 28 tools, cross-system data chaining, FastMCP servers, and external system integrations.

## 2. AI Agent Application

The agent leverages **OpenAI GPT-4o-mini** (configurable via `OPENAI_MODEL`) with **Pydantic AI** for native tool calling. The LLM autonomously decides which of the 28 registered tools to call, in what order, and how many times. Falls back to **Ollama** local models when no API key is available. A second **CS Assistant Agent** shares all tools with free-form text output for conversational monitoring via the Streamlit UI.

### 2.1 LLM-Powered Intelligence

- **Risk Analysis**: Evaluates API errors, violations, warnings → risk levels (low/medium/high/critical) with business impact and resolution time estimates.
- **Summary Generation**: Human-readable reports for CS teams, translating technical states to actionable insights.
- **Action Recommendations**: Prioritized steps with owner assignment (CS, Finance, IT/DevOps, Legal, Sales Ops).
- **Error Interpretation**: Converts technical codes (e.g., `INVALID_SESSION_ID`) to plain English with resolution steps.

### 2.2 Autonomous Actions

**(1)** Auto-provisions the SaaS tenant with tier-appropriate configuration (Enterprise/Growth/Starter).
**(2)** Creates a 14-task onboarding checklist with dependencies, ownership (system/cs_team/customer), and due dates.
**(3)** Sends Slack notifications to `#cs-onboarding-alerts` (blocked) or `#cs-onboarding` (success/escalate).
**(4)** Sends customer-facing welcome emails with tenant ID and login URL.
**(5)** Generates HTML/Markdown/JSON reports for auditing and traceability.
**(6)** Post-provisioning: monitors progress, detects risks, sends reminders, escalates stalled onboardings.
**(7)** Portfolio management: aggregated alerts across accounts, suggested actions with approve/dismiss, batch reminders.
**(8)** Remediation loop: the CS assistant can simulate source-system fixes (deal closed, contract executed, invoice paid, missing record created) and safely re-run onboarding from BLOCK/ESCALATE into PROCEED.
**(9)** Sentiment analysis: ML-powered scoring of customer interactions via DistilBERT (PyTorch + HuggingFace) with trend detection; runs as a mandatory step (step 5) after every provisioning alongside risk assessment; negative sentiment shifts health to at_risk and triggers proactive outreach suggestions.
**(10)** Product assistance: looks up tier configuration (features, limits, storage), SLA details, contractual terms, and implementation prerequisites from CLM and provisioning data.

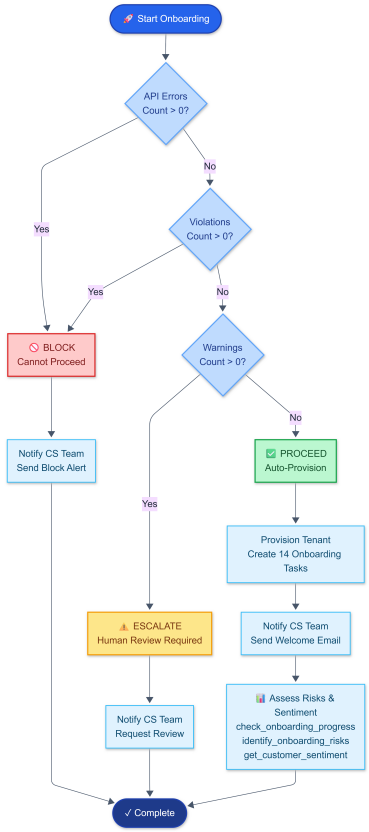


**Figure 2:** Decision: API Errors/Violations → BLOCK, Warnings → ESCALATE, Clear → PROCEED.

### 2.3 Onboarding Task Management

The provisioning system creates a 14-task onboarding checklist:

**Task Categories:**

- **Automated (4)**: Tenant creation, API credentials, welcome email, training materials
- **CS Team (5)**: Schedule/conduct kickoff, SSO config, custom reports, 30-day check-in
- **Customer (4)**: Verify login, complete tour, invite team, create first campaign
- **Milestone (1)**: Onboarding complete marker

**API Endpoints:**

- `GET /tasks/{id}` – All tasks with status
- `GET /tasks/{id}/pending?owner=cs_team` – Filtered pending
- `GET /tasks/{id}/overdue` – Overdue task alerts
- `PUT /tasks/{id}/{task_id}` – Update task status
- `GET /tasks/{id}/next-actions` – Ready-to-execute tasks

Tasks include dependencies (e.g., "Complete Tour" depends on "Verify Login"), due dates calculated relative to provisioning, and ownership assignment enabling filtered views per team.

### 2.4 Sentiment Analysis as a Proactive Risk Signal

Beyond task and billing state, the CS assistant also tracks **customer sentiment** from inbound and outbound interactions such as email, chat, support tickets, and calls. Sentiment scoring uses a **DistilBERT transformer model** (`distilbert-base-uncased-finetuned-sst-2-english`) via **PyTorch + HuggingFace Transformers** for real ML inference, with graceful fallback to keyword-based scoring when the ML stack is not installed. The model is lazy-loaded on first use to avoid impacting startup time. The prototype exposes two sentiment tools: `log_customer_interaction` records structured interactions for an account, and `get_customer_sentiment` returns a score, label (positive/neutral/negative), trend (improving/stable/declining), and the model used. This allows the agent to identify onboarding risk *before* a task is formally overdue or blocked.

| Sentiment Signal | Agent Behavior |
|---|---|
| Negative sentiment + declining trend | Recommend proactive outreach, schedule a check-in call, raise account priority in portfolio alerts |
| Negative sentiment + blocked onboarding tasks | Escalate urgency because operational blockers and customer frustration are compounding |
| Improving sentiment after outreach | Reduce risk posture and keep onboarding in the monitored PROCEED path |

### 2.5 Business Rule Validation (Invariants)

The agent enforces a two-tier validation system across five domains. **Violations** are blocking (cause BLOCK decision); **Warnings** are non-blocking (cause ESCALATE for human review).

| Domain | Blocking Violations | Non-blocking Warnings |
|---|---|---|
| **Account** | Missing Id/Name, IsDeleted=true | Missing BillingCountry, Industry, OwnerId |
| **Opportunity** | Stage $\neq$ "Closed Won", Missing Id/AccountId | Missing Amount, CloseDate, ContractId |
| **Contract (CLM)** | Status $\notin$ {EXECUTED, SIGNED}, Missing contract_id | No effective/expiry date, pending signatories |
| **Invoice** | Voided, Cancelled, Missing invoice_id | Overdue, Pending, <50% paid, missing due_date |
| **User** | Inactive, Missing Id/Email/ProfileId | Missing Title, Department, ManagerId |

## 3. Orchestration & Event-Driven Flows

| Trigger | Source | Endpoint |
|---|---|---|
| Webhook | Salesforce Flow | `POST /webhook/onboarding` |
| Manual | CS Team | `POST /demo/run/{account_id}` |
| Batch | Scheduler | `POST /demo/run-all` |

**Error Simulation**: `/demo/enable-random-errors` injects failures (401 auth, 400 validation, 429 rate limit, 500 server) at configurable rates for chaos testing without modifying business logic.

**Report Generation**: Each run produces three output files:

- `run_report_*.md` – Full Markdown report with all details
- `email_*.html` – Professional HTML email (blocked, escalation, success, or welcome)
- `audit_*.json` – Machine-readable audit log

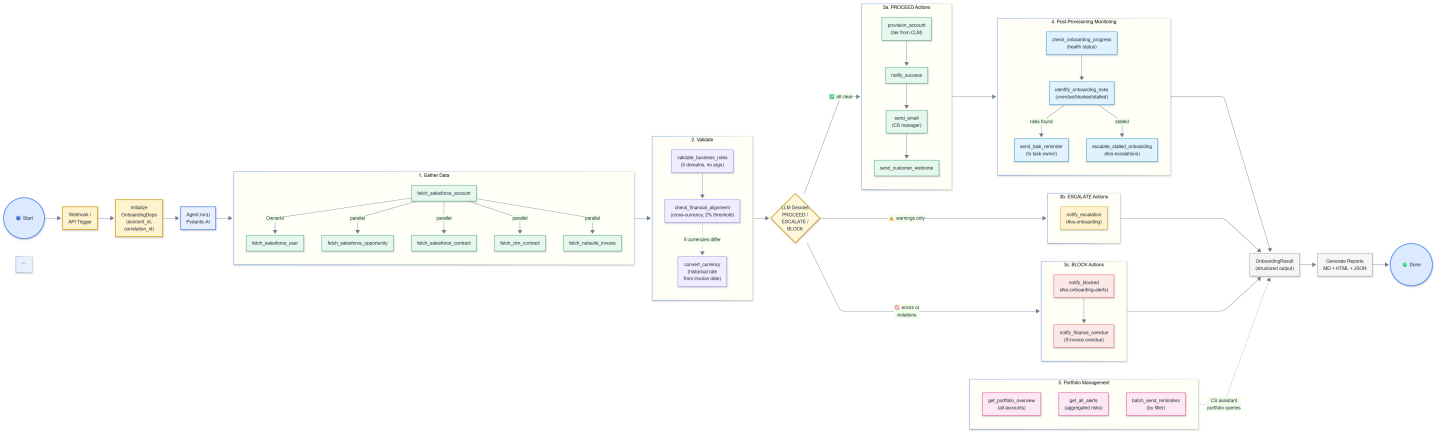


**Figure 3:** Agent workflow: full lifecycle from webhook ingestion through tool calling, validation, decision routing, post-provisioning monitoring, and portfolio management.

### 3.1 API Error Classification & Handling

Each integration returns structured error payloads with `error_type`, `error_code`, `http_status`, `message`, `resolution`, and `owner`:

| Error Type | HTTP | Resolution |
|---|---|---|
| Authentication | 401 | Re-authenticate, refresh token (IT/DevOps) |
| Authorization | 403 | Check profile/permission sets (System Admin) |
| Validation | 400 | Fix field values in source (Data Admin) |
| Rate Limit | 429 | Implement backoff, increase quota (Integration Admin) |
| Server Error | 500 | Check status page, retry later (Support Team) |

### 3.2 Error Handling Features

- **In-place Error Simulator**: `ERROR_SIMULATOR` object modified in-place when enabled; all modules reference the same instance
- **Comprehensive Catching**: Each integration catches specific error types AND generic `APIError` as fallback
- **Error-Aware Decisions**: `make_decision` checks `api_errors` first — system failures block onboarding
- **Full Traceability**: API errors added to violations and appear in reports with context

## 4. Trade-offs, Assumptions & Considerations

### 4.1 Key Trade-offs

| Decision | Trade-off |
|---|---|
| API Errors → BLOCK | Integrity over speed |
| Sequential Data Fetching | Clearer failure tracing but has higher latency |
| Rule-based Fallback | Less smart but reliable |
| 14 Fixed Tasks | Predictable but rigid |
| LLM Advisory Only | Auditable but less adaptive |

### 4.2 Assumptions

- Salesforce is source of truth for accounts
- CLM status reflects actual signatures
- Invoice payment status is current
- Cross-currency comparison uses historical ECB rates from invoice date with 2% threshold
- One invoice per account per onboarding run
- One Opportunity per Account (no multi-deal)
- Contract has a single version, no amendments
- CS monitors Slack channels for alerts
- Single onboarding per account at a time
- Customer email available in CLM signatories
- Account ID consistent across all systems

### 4.3 Limitations

- **Mock integrations**: Salesforce, NetSuite, CLM use simulated data; not connected to real APIs
- **In-memory state**: Provisioning and task data lost on restart; no persistent database
- **Single instance**: No horizontal scaling or load balancing
- **Limited approval UI**: Suggested Actions panel provides approve/dismiss and real backend execution; full Slack interactive approval with identity/policy controls not yet implemented
- **No webhook auth**: Inbound webhooks not verified via HMAC or shared secret

### 4.4 Testing & Demonstration

The system includes 10 pre-configured demo scenarios validating all decision paths. Three PROCEED scenarios include post-provisioning simulation to demonstrate proactive risk detection, and five BLOCK/ESCALATE scenarios now support remediation plus onboarding re-run to demonstrate closed-loop CS intervention:

| Account ID | Scenario | Description | Decision |
|---|---|---|---|
| ACME-001 | Happy Path | All checks pass, contract executed, invoice paid | PROCEED |
| BETA-002 | Opportunity Blocked | Opportunity in "Negotiation" stage | BLOCK |
| GAMMA-003 | Overdue Invoice | Invoice 25+ days overdue, needs finance review | ESCALATE |
| DELETED-004 | Deleted Account | Account marked `IsDeleted=true` | BLOCK |
| MISSING-999 | Account Not Found | Account does not exist in any system | BLOCK |
| FOREX-005 | FX Invoice Mismatch | CAD $145k invoice vs USD $100k opportunity; historical rate gap >2% | ESCALATE |
| PARTIAL-006 | Partial Payment Gap | $190k paid of $200k invoice (5% underpayment exceeds 2% threshold) | ESCALATE |
| STARTER-007 | Customer Not Logged In | All checks pass; post-provisioning: customer hasn't logged in after 5 days | PROCEED |
| GROWTH-008 | Stalled Onboarding | All checks pass; post-provisioning: <30% complete after 10 days, kickoff not done | PROCEED |
| ENTERPRISE-009 | SSO & Blocked Tasks | All checks pass; post-provisioning: SSO blocked, multiple tasks overdue after 8 days | PROCEED |

**Risk and action scenarios after provisioning:**

- `STARTER-007`: detects customer inactivity (no login after 5 days) and recommends or executes a login reminder.
- `GROWTH-008`: detects onboarding stalling (<30% complete after 10 days) and escalates to CS management.
- `ENTERPRISE-009`: detects blocked SSO work plus overdue customer tasks and recommends SSO follow-up, task reminders, or escalation.
- `Sentiment scenario`: repeated negative customer interactions produce a declining sentiment trend; the CS assistant recommends a proactive call and the suggested-action flow can execute that outreach.
- `BETA-002`, `DELETED-004`, `MISSING-999`, `FOREX-005`, `PARTIAL-006`: support simulated remediation of source-system blockers/warnings followed by a fresh onboarding run into provisioning.

### 4.5 Observability & Tracing

The agent supports **dual opt-in tracing** — both can run simultaneously:

- **Pydantic Logfire**: Native agent tracing — reasoning chains, tool calls with inputs/outputs, structured output validation.
- **LangSmith**: Agent execution runs, tool invocations, LLM completions via OpenTelemetry bridge (`OtelSpanProcessor`). Logfire owns the TracerProvider; LangSmith attaches as a span processor.
- **Structured JSON Logging**: All events logged with `correlation_id`, timestamps, and context.

### 4.6 Security Patterns

All integrations are mocked but demonstrate production security patterns:

- **OAuth Simulation**: Token expiry and refresh flows with distinct error types for expired vs invalid tokens
- **Permission Checking**: Role-based access control verified before every API call
- **Credential Validation**: Structured error hierarchy (`AuthenticationError`, `AuthorizationError`, `ValidationError`, `RateLimitError`)
- **Audit Logging**: Correlation IDs propagated end-to-end for traceability across all systems
- **Error Masking**: No sensitive data (tokens, secrets) exposed in API responses or reports

## 5. Multi-Agent Architecture via Model Context Protocol (MCP) Server Connections

*Note: All 28 agent tools are mirrored as FastMCP server definitions in `app/mcp/` (8 servers: Salesforce, CLM, NetSuite, Currency, Provisioning, Notifications, Validation, Sentiment). These can be extracted to standalone MCP services for multi-agent collaboration.*

The architecture supports multi-agent collaboration through the **Model Context Protocol (MCP)**, enabling specialized agents to interact through standardized, governed tool interfaces rather than direct API access. Each integration is wrapped as a FastMCP server, cleanly separating decision logic from execution concerns.

| Agent | Responsibility & MCP Tools |
|---|---|
| Coordinator | Orchestrates end-to-end workflow (`salesforce.*`, `provision.*`) |
| Contract Agent | Monitors contract execution and signatures (`clm.get_contract`) |
| Finance Agent | Evaluates billing, payments, and cross-currency alignment (`netsuite.*`, `currency.*`) |
| Task Monitor | Detects overdue onboarding tasks (`tasks.get_overdue`) |
| Sentiment Agent | Scores customer interactions via ML inference (`sentiment.*`) |

**Benefits**: Independent versioning per integration • Clear domain ownership • Centralized access control and audit logging • Scalable agent complexity • Cross-agent context sharing.

**Concurrency & API Execution Strategy**: MCP servers act as controlled execution layers for external APIs, enabling concurrent and batch-aware request handling. Batch or composite API calls are leveraged where supported (e.g., Salesforce composite reads), with fallback to bounded parallel single-entity requests. Concurrency limits, rate limiting, and circuit breakers are enforced at the MCP layer to prevent downstream overload and cascading failures.

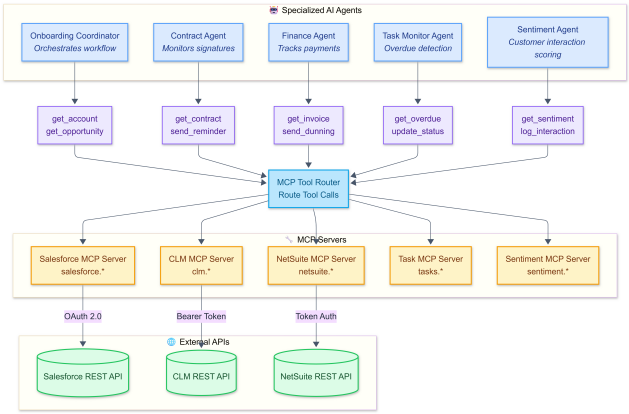


**Figure 4:** MCP: Specialized agents collaborate through shared execution servers wrapping external APIs.

## 6. Production Considerations

*For additional production enhancements, see the `production_roadmap` directory in the repository.*

### Cloud Deployment & CI/CD

**Frontend**: Streamlit UI with four pages (Dashboard, Portfolio, Run Onboarding, Chat). Production: migrate to React/ASP.NET with auth and WebSocket updates.
**Infrastructure**: ECS/Fargate with horizontal scaling. **AWS Bedrock** for multi-model LLM inference (Claude, Llama, Titan) with IAM auth. Docker + Kubernetes for consistent environments, auto-scaling, and zero-downtime deployments.
**CI/CD**: GitHub Actions / CodePipeline — unit tests for invariants, risk analysis, and decision routing on every PR. Reproducible, auditable deployments with safe rollback.

**Observability**: Dual tracing (Pydantic Logfire + LangSmith via OpenTelemetry). Prometheus metrics, DataDog/Jaeger distributed tracing, PagerDuty alerts.

### Agentic Execution & API Governance

**Deterministic Boundaries**: LLM reasons through tools via `@agent.tool`, but `validate_business_rules` and `check_financial_alignment` define all violations/warnings. LLM routes (PROCEED/ESCALATE/BLOCK) based on these outputs.
**Data Chaining**: Fetch account → user/opportunity in parallel → sequential chain: Opp.ContractId → SF Contract → CLM → Invoice. Fallback to account-based lookups. Production: batch APIs, concurrent fetches, bounded backoff.
**Idempotency & Retry**: Runs keyed by `correlation_id` for safe replays. Retryable errors (5xx, 429) → exponential backoff; non-retryable → immediate halt. Circuit breakers prevent cascading failures.

**Governance**: PII-masked data sent to LLMs. Per-source rate limits. Agent version, rule set, and prompt version recorded per run. All 28 tools mirrored as 8 FastMCP servers, extractable to standalone services with A2A protocol.